UNIVERSIDAD POLITÉCNICA DE SAN LUIS POTOSÍ
Ingeniería en Tecnologías de la Información

CNO IV — REPORTE DE LABORATORIO

File Path Traversal: Validación de extensión con bypass de byte nulo

Actividad 12 — Road to Hall of Fame

PortSwigger Web Security Academy
portswigger.net/web-security/file-path-traversal/lab-validate-file-extension-null-byte-bypass

---

Alumno: Jorge Abraham Castelo Curioca

Materia: CNO IV

Docente: Mtro. Servando López Contreras

Periodo: Segundo Parcial

Documento: Path Traversal 6 Actividad 12.pdf

1. Ficha técnica del laboratorio

Nombre del laboratorio	File path traversal, validation of file extension with null byte bypass
Nivel	Practitioner
Tipo de explotación	File Path Traversal / Directory Traversal — evasión de validación de extensión mediante inyección de byte nulo (`%00`)
Objetivo del laboratorio	Evadir la validación que exige que el parámetro `filename` termine con la extensión permitida, con el fin de recuperar el contenido del archivo de sistema `/etc/passwd`.

2. Explicación técnica de la vulnerabilidad

Para restringir el acceso a recursos no autorizados, la aplicación verifica que el nombre de archivo recibido en el parámetro `filename` **termine** con una extensión permitida (en este laboratorio, `.png`). Esta validación, por sí sola, no impide que el valor incluya además secuencias de salto de directorio `../`, siempre que la cadena completa conserve la extensión exigida al final.

La vulnerabilidad de **Null Byte Bypass** surge de una asimetría entre la forma en que el lenguaje de alto nivel de la aplicación web trata las cadenas de texto y la forma en que las bibliotecas de bajo nivel del sistema operativo (heredadas de C/C++) interpretan un byte nulo. Mientras el validador de la aplicación evalúa la cadena completa y comprueba correctamente que termina en `.png`, la llamada al sistema de archivos que finalmente abre el recurso interpreta el carácter nulo (`0x00`, codificado en la URL como `%00`) como un **terminador de cadena**, descartando todo el contenido posterior a ese byte. De esta manera, es posible construir una ruta que, a ojos del validador, termina en la extensión permitida, pero que el sistema operativo trunca justo antes de dicho sufijo, abriendo en realidad el archivo indicado por la parte anterior al byte nulo. Esta vulnerabilidad corresponde a la categoría **A01: Broken Access Control** de OWASP Top 10 y compromete directamente la confidencialidad de la información del servidor.

3. Procedimiento paso a paso

1. **Interceptación de la petición de imagen.** Se inició el laboratorio y, mediante Burp Suite configurado como proxy del navegador, se revisó la pestaña Proxy / Intercept mientras se navegaba por el catálogo de productos. Se identificaron las peticiones `GET /image?filename=<n>.jpg` que cargan cada imagen, confirmando que el parámetro `filename` es la entrada controlada por el usuario.

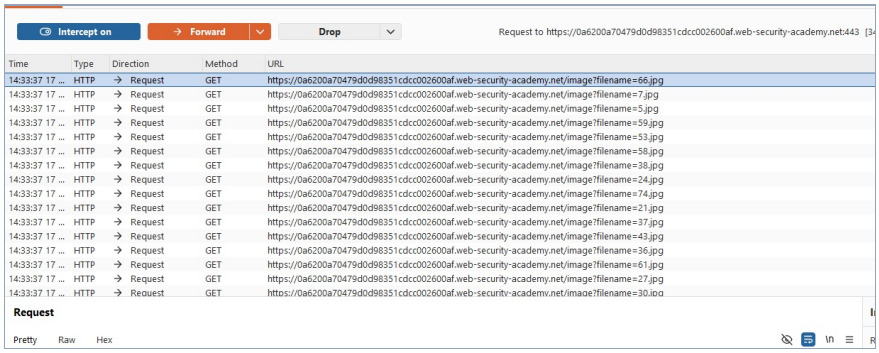


Figura 1. Burp Suite – Proxy/Intercept: peticiones GET a /image?filename=<n>.jpg generadas al cargar el catálogo de productos.

2. **Análisis de la petición base en Repeater.** Se envió al Repeater una petición normal (`filename=58.jpg`) para confirmar el comportamiento de referencia: el servidor responde `200 OK` con `Content-Type: image/jpeg` y el contenido binario de la imagen solicitada, confirmando que el archivo se sirve a partir del valor recibido en `filename`.

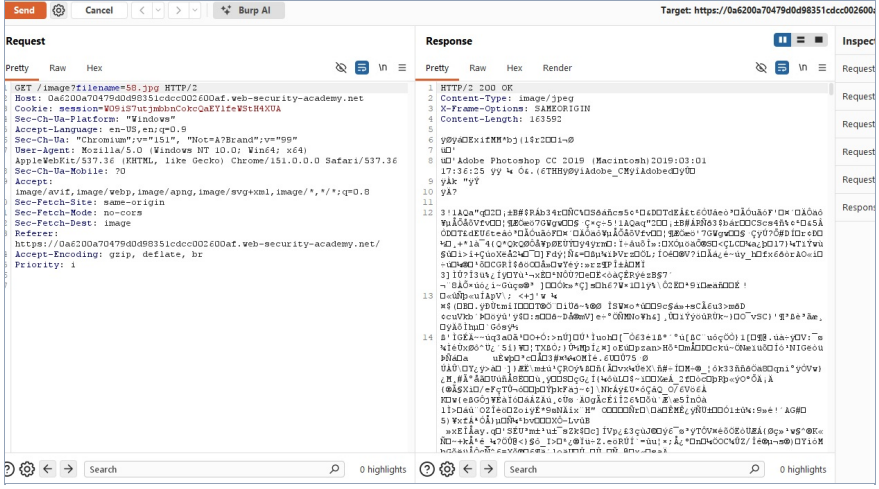


Figura 2. Repeater: solicitud de referencia con filename=58.jpg y respuesta 200 OK devolviendo la imagen en formato binario.

3. **Construcción y envío del payload con byte nulo.** Al intentar enviar únicamente `../../../etc/passwd`, la aplicación rechaza la petición por no terminar con la extensión exigida. Para satisfacer esa validación y, a la vez, truncar la ruta en el sistema de archivos, se añadió un byte nulo codificado en URL (`%00`) seguido de la extensión requerida (`.png`, según se confirmó en la respuesta del servidor):

```
filename=../../../etc/passwd%00.png
```

La petición modificada se envió desde el Repeater hacia el servidor para verificar si la validación de extensión era evadida.

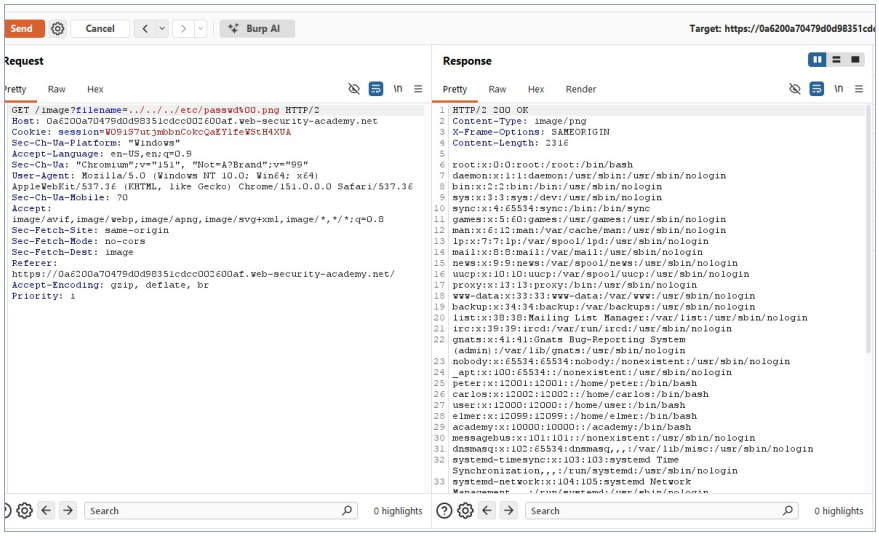


Figura 3. Repeater: solicitud con el payload ../../../etc/passwd%00.png y respuesta 200 OK devolviendo en texto plano el contenido completo de /etc/passwd.

4. **Confirmación de la explotación.** El validador de la aplicación aprobó la cadena por terminar en `.png`, mientras que el sistema de archivos truncó la lectura en el byte nulo y devolvió el contenido íntegro de `/etc/passwd`. La plataforma de PortSwigger Web Security Academy marcó el laboratorio como resuelto, confirmando de manera oficial el éxito de la explotación.

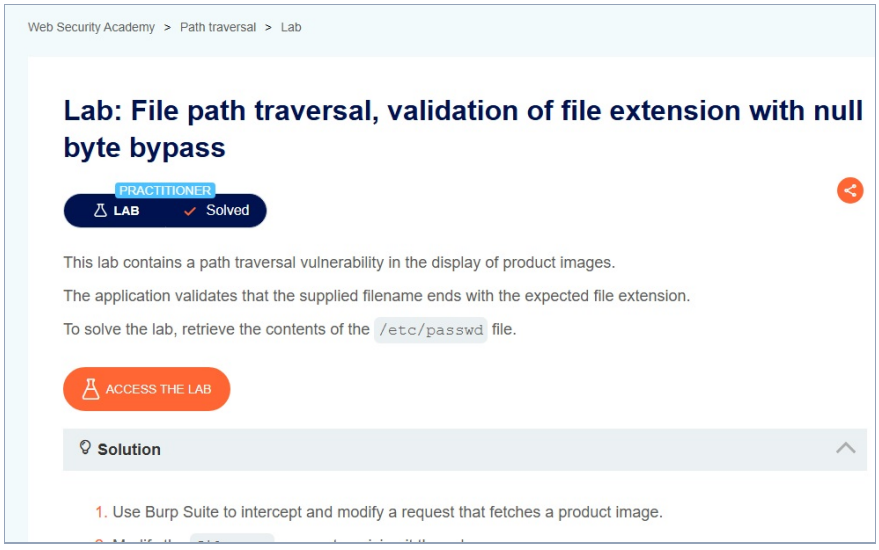


Figura 4. Confirmación "Lab Solved" otorgada por PortSwigger Web Security Academy tras la explotación exitosa.

4. Explicación del payload

```
../../../etc/passwd%00.png
```

El payload combina dos técnicas que actúan en capas distintas del sistema. La parte `../../../etc/passwd` es la ruta de traversal clásica que, al resolverse, apunta al archivo objetivo fuera del directorio de imágenes. El fragmento `%00` es la codificación en URL del byte nulo (carácter `0x00`); al decodificarse, el validador de alto nivel de la aplicación recibe la cadena completa `../../../etc/passwd[NUL].png` y, dado que termina en `.png`, la considera válida y la deja pasar. Sin embargo, cuando el sistema operativo abre efectivamente el archivo, interpreta el byte nulo como el final real de la cadena y descarta todo lo que viene después, por lo que en la práctica abre `../../../etc/passwd` y no un archivo `.png`. Esta discrepancia entre lo que valida la aplicación y lo que ejecuta el sistema operativo es la esencia del bypass.

5. Mitigación recomendada

- Rechazar de forma explícita cualquier entrada que contenga bytes nulos (`\0` o `%00`) antes de procesar el nombre de archivo, y mantener actualizados los entornos de ejecución, ya que las versiones modernas de los lenguajes más comunes ya detectan y bloquean este carácter en rutas de archivo.
- No basar la validación únicamente en la extensión del archivo: verificar también el nombre completo contra una lista blanca de caracteres permitidos (alfanuméricos y un punto para la extensión), rechazando cualquier otro carácter de control.
- Canonicalizar la ruta resultante mediante las APIs del sistema operativo (por ejemplo, `getCanonicalPath()`) y confirmar que el archivo final resida dentro del directorio base permitido antes de abrirlo.
- Ejecutar el proceso que sirve los archivos con el mínimo privilegio necesario, para limitar el impacto si la validación vuelve a fallar.